

JavaScript

```
/**  
 * @author Rocco Mazzeo  
 * @email rocco.mazzeo@gmail.com  
 * @linkedin https://www.linkedin.com/in/roccomazzeo  
 */
```

Introduzione a JavaScript

JavaScript è un linguaggio di programmazione nato nel 1995 per aggiungere interattività alle pagine web. Creato da Brendan Eich, è diventato uno standard del web, supportato da tutti i principali browser. JavaScript permette di manipolare il DOM, gestire eventi e comunicare con server tramite AJAX.

Statistiche di utilizzo dei linguaggi di programmazione (2004-2024)

- Negli ultimi 20 anni, la popolarità dei linguaggi di programmazione è cambiata notevolmente.
- **JavaScript** è diventato il linguaggio più usato per lo sviluppo web.
- **Python** ha visto una crescita enorme grazie a data science, AI e automazione.
- **Java** e **C#** rimangono molto diffusi nello sviluppo enterprise.
- **PHP** ha perso terreno ma è ancora usato per il web.
- **C/C++** sono stabili, soprattutto in ambiti di sistema e embedded.

- **Top 5 linguaggi per il web nel 2025:**
 - i. JavaScript
 - ii. TypeScript
 - iii. Python
 - iv. PHP
 - v. Java
- **JavaScript e TypeScript** dominano lo sviluppo web moderno, seguiti da Python (soprattutto per backend e API), PHP (ancora molto usato per siti e CMS), e Java (popolare per applicazioni enterprise).

Sintassi base di JavaScript in HTML

Per includere JavaScript in una pagina HTML, si utilizza il tag `<script>` :

```
<!DOCTYPE html>
<html>
  <head>
    <title>Esempio JavaScript</title>
  </head>
  <body>
    <h1>Ciao JavaScript!</h1>
    <script>
      document.querySelector("h1").textContent = "Benvenuto in JavaScript!";
    </script>
  </body>
</html>
```

Come funziona JavaScript nel browser

- JavaScript viene eseguito direttamente dal browser, lato client.
- Può manipolare la pagina web (DOM), rispondere agli eventi e comunicare con server.
- Ogni browser ha un motore JavaScript (es. V8 per Chrome, SpiderMonkey per Firefox).

Prova pratica: Console del browser

1. Apri una pagina web qualsiasi.
2. Premi `F12` o `Ctrl+Shift+I` per aprire gli strumenti di sviluppo.
3. Vai alla scheda "Console".
4. Scrivi e esegui codice JavaScript, ad esempio:

```
console.log("Ciao dal browser!");
```

Hello World online: prova JavaScript senza installare nulla

- Puoi provare JavaScript direttamente online senza installare nulla, usando editor come:
 - [JSFiddle](#)
 - [CodePen](#)
 - [PlayCode](#)
 - [JSBin](#)

Esempio di Hello World:

```
console.log("Hello World!");
```

- Incolla questo codice nell'editor JavaScript del sito scelto e premi "Run" o

Inserire JavaScript in un file esterno

- È buona pratica separare il codice JavaScript dall'HTML.
- Si crea un file, ad esempio `script.js`, e lo si collega all'HTML:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Esempio con file esterno</title>
    <script src="script.js"></script>
  </head>
  <body>
    <h1>Test JavaScript esterno</h1>
  </body>
</html>
```

- Il browser carica ed esegue il file JavaScript indicato.

Tipi di dati in JavaScript

- **Stringhe:** sequenze di caratteri, es. `"Ciao"`
- **Numeri:** sia interi che decimali, es. `42` , `3.14`
- **Booleani:** `true` o `false`
- **Null:** valore intenzionalmente assente
- **Undefined:** variabile dichiarata ma non inizializzata
- **Oggetti:** collezioni di coppie chiave-valore
- **Array:** liste ordinate di valori

Operatori principali

- **Aritmetici:** `+`, `-`, `*`, `/`, `%`
- **Assegnamento:** `=`, `+=`, `-=`, `*=`, `/=`
- **Confronto:** `==`, `===`, `!=`, `!==`, `<`, `>`, `<=`, `>=`
- **Logici:** `&&` (and), `||` (or), `!` (not)
- Nota sugli operatori `===` e `!==`:
 - L'operatore `===` (strettamente uguale) confronta sia il valore che il tipo delle due espressioni. Restituisce `true` solo se entrambi sono uguali e dello stesso tipo.
- Esempio: `5 === '5'` restituisce `false`, mentre `5 === 5` restituisce `true`.

- ○ L'operatore `!==` (strettamente diverso) confronta sia il valore che il tipo delle due espressioni. Restituisce `true` solo se sono diversi o di tipo diverso.
- Esempio: `5 !== '5'` restituisce `true`, mentre `5 !== 5` restituisce `false`.
- Questi operatori sono preferibili rispetto a `==` e `!=` perché evitano conversioni implicite di tipo rendendo il confronto più prevedibile e sicuro.

Esempi di operatori

```
let a = 5;  
let b = 2;  
console.log(a + b); // 7  
console.log(a === b); // false  
console.log(a > b && b > 0); // true
```

Strutture di controllo: if/else

```
let eta = 18;  
if (eta >= 18) {  
  console.log("Sei maggiorenne");  
} else {  
  console.log("Sei minorenn");  
}
```

Strutture di controllo: switch

```
let giorno = "lunedì";  
switch (giorno) {  
  case "lunedì":  
    console.log("Inizio settimana");  
    break;  
  case "venerdì":  
    console.log("Quasi weekend!");  
    break;  
  default:  
    console.log("Giorno normale");  
}
```

Cicli: for

```
for (let i = 0; i < 5; i++) {  
    console.log("Valore di i:", i);  
}
```


Cicli: while e do...while

```
let count = 0;
while (count < 3) {
  console.log(count);
  count++;
}
```

```
let x = 0;
do {
  console.log(x);
  x++;
} while (x < 3);
```

Funzioni in JavaScript

```
function somma(a, b) {  
    return a + b;  
}  
console.log(somma(3, 4)); // 7
```

Funzioni arrow (funzioni freccia)

```
const moltiplica = (x, y) => x * y;  
console.log(moltiplica(2, 5)); // 10
```

- Questa funzione è definita come arrow function (funzione freccia), una sintassi introdotta in ES6 che consente di scrivere funzioni in modo più conciso.
- Le arrow function non hanno il proprio `this`, `arguments`, `super` o `new.target`, e sono spesso utilizzate per funzioni anonime o callback.
- Utilizzare le arrow function può rendere il codice più leggibile e mantenere il contesto di `this` dalla funzione esterna.

Esercizio: Funzioni classiche vs arrow function

Scrivi due funzioni che calcolano il quadrato di un numero:

1. Una funzione classica chiamata `quadratoClassico`.
2. Una arrow function chiamata `quadratoFreccia`.

Entrambe devono restituire il quadrato del valore passato come argomento.

Prova a chiamare entrambe le funzioni con il valore `5` e stampa il risultato in console.

Soluzione

```
function quadratoClassico(x) {  
    return x * x;  
}  
  
const quadratoFreccia = (x) => x * x;  
  
console.log(quadratoClassico(5)); // 25  
console.log(quadratoFreccia(5)); // 25
```

Differenza del `this` tra funzioni classiche e arrow function

- **Funzioni classiche:** il valore di `this` dipende da come la funzione viene chiamata. Se usata come metodo di un oggetto, `this` fa riferimento all'oggetto stesso.

```
const persona = {  
  nome: "Luca",  
  saluta: function () {  
    console.log(this.nome); // 'Luca'  
  },  
};  
persona.saluta(); // 'Luca'
```

- **Arrow function:** non hanno il proprio `this`. Ereditano il `this` dal contesto esterno in cui sono state definite.

```
const persona = {  
  nome: "Luca",  
  saluta: () => {  
    console.log(this.nome); // undefined (non è l'oggetto persona)  
  },  
};  
persona.saluta(); // undefined
```

Nota: Usa funzioni classiche per i metodi degli oggetti quando serve accedere a `this`. Le arrow function sono utili per callback e funzioni anonime dove non serve un proprio `this`.

Oggetti e proprietà

```
const persona = {  
  nome: "Luca",  
  eta: 30,  
};  
console.log(persona.nome); // 'Luca'
```

Nota: In JavaScript, gli oggetti sono strutture fondamentali che permettono di raggruppare dati e funzionalità. Ogni oggetto è una collezione di coppie chiave-valore, dove le chiavi sono stringhe (o simboli) e i valori possono essere di qualsiasi tipo, incluse altre funzioni (metodi).

Approfondimento: Oggetti JavaScript

- Gli oggetti possono essere creati usando la notazione letterale `{}` oppure tramite costruttori come `new Object()`.
- È possibile aggiungere, modificare o rimuovere proprietà anche dopo la creazione dell'oggetto.
- Le proprietà possono essere accedute sia con la notazione punto (`oggetto.proprieta`) sia con la notazione con parentesi quadre (`oggetto['proprieta']`).

Esempio: aggiunta e accesso a proprietà e metodi

```
const persona = {  
  nome: "Luca",  
  eta: 30,  
  saluta: function () {  
    console.log("Ciao, sono " + this.nome);  
  },  
};  
  
persona.cognome = "Rossi"; // aggiunta di una nuova proprietà  
console.log(persona["cognome"]); // accesso tramite parentesi quadre  
persona.saluta(); // chiamata di un metodo
```

- Gli oggetti sono alla base di molte strutture in JavaScript, inclusi array, funzioni e persino il DOM.
- Comprendere bene gli oggetti è essenziale per scrivere codice JavaScript efficace e modulare

Array e metodi principali

```
const numeri = [1, 2, 3, 4];  
numeri.push(5); // aggiunge 5 alla fine  
console.log(numeri.length); // 5  
console.log(numeri[0]); // 1
```

Le principali funzioni JavaScript integrate

JavaScript offre molte funzioni integrate (built-in) che facilitano la manipolazione di stringhe, numeri, array e altro.

Funzioni per le stringhe

- **length** : restituisce la lunghezza della stringa

```
let testo = "Ciao";  
console.log(testo.length); // 4
```

- **toUpperCase()** / **toLowerCase()** : converte la stringa in maiuscolo/minuscolo

```
console.log(testo.toUpperCase()); // "CIAO"  
console.log(testo.toLowerCase()); // "ciao"
```

- **includes()** : verifica se una sottostringa è presente

```
console.log(testo.includes("ia")); // true
```

- **replace()** : sostituisce una parte della stringa

```
let nuovoTesto = testo.replace("C", "B");  
console.log(nuovoTesto); // "Biao"
```

Funzioni per i numeri

- `parseInt()` / `parseFloat()` : converte una stringa in numero intero/decimale

```
let numero = parseInt("42");  
let decimale = parseFloat("3.14");
```

- `Math.round()` : arrotonda al numero intero più vicino

```
console.log(Math.round(3.7)); // 4
```

- **Math.random()** : genera un numero casuale tra 0 e 1

```
console.log(Math.random());
```

- **Math.max()** / **Math.min()** : restituisce il massimo/minimo tra i valori

```
console.log(Math.max(1, 5, 3)); // 5
```


Funzioni per gli array

- **push()** / **pop()** : aggiunge/rimuove elementi in fondo all'array

```
let arr = [1, 2, 3];  
arr.push(4); // [1,2,3,4]  
arr.pop(); // [1,2,3]
```

- **shift()** / **unshift()** : rimuove/aggiunge elementi all'inizio

```
arr.unshift(0); // [0,1,2,3]  
arr.shift(); // [1,2,3]
```

- **map()** : crea un nuovo array applicando una funzione a ogni elemento

```
let quadrati = arr.map((x) => x * x); // [1,4,9]
```

- **filter()** : filtra gli elementi secondo una condizione

```
let pari = arr.filter((x) => x % 2 === 0); // [2]
```

- **forEach()** : esegue una funzione per ogni elemento

```
arr.forEach((x) => console.log(x));
```

Funzioni per la manipolazione degli oggetti

- **Object.keys()** : restituisce un array con le chiavi dell'oggetto

```
const persona = { nome: "Anna", eta: 28 };  
console.log(Object.keys(persona)); // ["nome", "eta"]
```

- **Object.values()** : restituisce un array con i valori

```
console.log(Object.values(persona)); // ["Anna", 28]
```

- **Object.entries()** : restituisce array di coppie chiave-valore

```
console.log(Object.entries(persona)); // [["nome","Anna"],["eta",28]]
```

Funzioni utili varie

- **alert()** : mostra un messaggio popup

```
alert("Ciao!");
```

- **prompt()** : chiede un input all'utente

```
let nome = prompt("Come ti chiami?");
```

- **console.log()** : stampa in console

```
console.log("Messaggio di debug");
```

Esempio pratico: uso combinato di funzioni

```
let nomi = ["Anna", "Luca", "Marco"];  
let nomiMaiuscoli = nomi.map((n) => n.toUpperCase());  
console.log(nomiMaiuscoli); // ["ANNA", "LUCA", "MARC0"]
```

Sintassi base di JavaScript

- **Dichiarazione di variabili:**

- `var` è la parola chiave storica per dichiarare variabili. Ha ambito di funzione e può essere ridefinita.
- Oggi si preferiscono `let` e `const` per una gestione più sicura dell'ambito delle variabili.

```
var messaggio = "Ciao!";  
let nome = "Mario";  
const eta = 25;
```

Scope di var, let e const

- **var** : ha scope di funzione. Se dichiarata dentro una funzione, è visibile solo all'interno di essa; se dichiarata fuori, è globale. Non rispetta il blocco (`if` , `for` , ecc.).
- **let** : ha scope di blocco. È visibile solo all'interno del blocco in cui è dichiarata (ad esempio dentro `{ ... }`).
- **const** : come `let` , ha scope di blocco. Il valore non può essere riassegnato, ma se è un oggetto o array, le sue proprietà possono essere modificate.

Esempio:

```
if (true) {  
  var x = 1;  
  let y = 2;  
  const z = 3;  
}  
console.log(x); // 1 (var esce dal blocco)  
console.log(y); // ReferenceError  
console.log(z); // ReferenceError
```

```
let nome = "Mario";  
const eta = 25;
```


Hoisting in JavaScript

- **Hoisting** è il comportamento per cui le dichiarazioni di variabili e funzioni vengono "spostate" in cima al loro scope prima dell'esecuzione del codice.
- Solo le dichiarazioni vengono hoistate, non le assegnazioni di valore.

Hoisting con `var`

```
console.log(a); // undefined  
var a = 5;  
console.log(a); // 5
```

provate a stampare il valore di una variabile non definita e di una non assegnata

Hoisting con `let` e `const`

```
console.log(b); // ReferenceError  
let b = 10;
```

- Le variabili dichiarate con `let` e `const` non sono accessibili prima della dichiarazione: generano un errore (TDZ - Temporal Dead Zone).

Hoisting delle funzioni

```
saluta(); // "Ciao!"  
function saluta() {  
    console.log("Ciao!");  
}
```

- Le dichiarazioni di funzione vengono completamente hoistate (sia nome che corpo).

Hoisting delle function expression

```
saluta(); // TypeError: saluta is not a function
var saluta = function () {
  console.log("Ciao!");
};
```

- Solo la dichiarazione della variabile viene hoistata (`var saluta`), ma non l'assegnazione della funzione. Il comportamento è identico a quello delle variabili.

Nota: Comprendere l'hoisting aiuta a evitare errori e comportamenti inattesi nel codice JavaScript.

Challenge 1: Dichiarazione di variabili

Scrivi del codice che dichiara una variabile `nome` usando `let` e le assegna il valore `"Luca"`. Poi prova a dichiarare di nuovo la stessa variabile nello stesso blocco. Cosa succede?

Soluzione Challenge 1

```
let nome = "Luca";  
let nome = "Marco"; // Errore: Identifier 'nome' has already been declared
```

- **Errore:** Non puoi dichiarare due volte una variabile con `let` nello stesso blocco.

Challenge 2: Uso di `var` e riassegnazione

Dichiara una variabile `x` con `var` e assegnale il valore `10`. Poi riassegna `x` a `20` e stampa il risultato.

Soluzione Challenge 2

```
var x = 10;  
x = 20;  
console.log(x); // 20
```

- Con `var` puoi riassegnare il valore della variabile.

Challenge 3: Scope di blocco

Cosa succede se dichiarare una variabile con `let` dentro un blocco `if` e provi ad accedervi fuori dal blocco?

Soluzione Challenge 3

```
if (true) {  
  let y = 5;  
}  
console.log(y); // ReferenceError: y is not defined
```

- **Errore:** `let` ha scope di blocco, quindi `y` non è visibile fuori dall' `if`.

Challenge 4: Uso di `const`

Dichiara una variabile `PI` con `const` e assegnale il valore `3.14`. Prova a cambiare il valore di `PI`. Cosa succede?

Soluzione Challenge 4

```
const PI = 3.14;  
PI = 3.1416; // TypeError: Assignment to constant variable.
```

- **Errore:** Non puoi riassegnare una variabile dichiarata con `const`.

Challenge 5: Hoisting con `var` e `let`

Cosa succede se provi a usare una variabile dichiarata con `var` o `let` prima della sua dichiarazione?

Soluzione Challenge 5

```
console.log(a); // undefined (hoisting con var)
var a = 10;

console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 20;
```

- Le variabili `var` vengono "hoistate" (sollevate) ma inizializzate a `undefined`.
- Le variabili `let` (e `const`) non possono essere usate prima della dichiarazione.

Esercizi di base con JavaScript puro

- 1. Somma di numeri in un array**
- 2. Trova il valore massimo in un array**
- 3. Conta quante volte appare una lettera in una stringa**
- 4. Crea un nuovo array con i quadrati dei numeri**

```
const numeri = [1, 2, 3, 4, 5];  
let somma = 0;  
for (let i = 0; i < numeri.length; i++) {  
    somma += numeri[i];  
}  
console.log("Somma:", somma); // 15
```



```
const valori = [10, 5, 8, 21, 3];  
let max = valori[0];  
for (let i = 1; i < valori.length; i++) {  
    if (valori[i] > max) {  
        max = valori[i];  
    }  
}  
console.log("Massimo:", max); // 21
```

```
const frase = "javascript è fantastico";
const lettera = "a";
let conta = 0;
for (let i = 0; i < frase.length; i++) {
    if (frase[i] === lettera) {
        conta++;
    }
}
console.log(`La lettera '${lettera}' appare ${conta} volte.`);
```

```
const numeri = [1, 2, 3, 4];  
const quadrati = [];  
for (let i = 0; i < numeri.length; i++) {  
    quadrati.push(numeri[i] * numeri[i]);  
}  
console.log(quadrati); // [1, 4, 9, 16]
```

Accedere agli elementi HTML

Principali funzioni per accedere agli elementi HTML

- **getElementById(id)**

Seleziona il primo elemento con l'id specificato.

```
const titolo = document.getElementById("titolo");
```

- **getElementsByClassName(className)**

Restituisce una HTMLCollection di tutti gli elementi con la classe specificata.

```
const paragrafi = document.getElementsByClassName("paragrafo");
```

- **getElementsByTagName(tag)**

Restituisce una HTMLCollection di tutti gli elementi con il tag specificato.

```
const liste = document.getElementsByTagName("ul");
```

- **querySelector(selector)**

Restituisce il primo elemento che corrisponde al selettore CSS.

```
const primoParagrafo = document.querySelector(".paragrafo");
```

- **querySelectorAll(selector)**

Restituisce una NodeList di tutti gli elementi che corrispondono al selettore CSS.

```
const tuttiLi = document.querySelectorAll("li");
```

Ciclare sugli elementi HTML

- Con `for` classico (HTMLCollection o NodeList):

```
const elementi = document.getElementsByClassName("paragrafo");
for (let i = 0; i < elementi.length; i++) {
  console.log(elementi[i].textContent);
}
```

- Con `forEach` (NodeList da `querySelectorAll`):

```
const elementi = document.querySelectorAll("li");
elementi.forEach(function (el) {
  el.style.color = "blue";
});
```

- Convertire HTMLCollection in Array per usare `forEach` :

```
const paragrafi = document.getElementsByClassName("paragrafo");  
Array.from(paragrafi).forEach((p) => p.classList.add("evidenziato"));
```

Esempio pratico: evidenziare tutti i paragrafi

```
const paragrafi = document.querySelectorAll("p");
paragrafi.forEach((paragrafo) => {
  paragrafo.style.backgroundColor = "yellow";
});
```

- **Selezionare elementi:**

```
// Seleziona il primo elemento con id "titolo"
const titolo = document.getElementById("titolo");

// Seleziona il primo elemento con classe "paragrafo"
const paragrafo = document.querySelector(".paragrafo");

// Seleziona tutti gli elementi <li>
const elementiLista = document.querySelectorAll("li");
```


Modificare il testo e gli attributi di un elemento

- Cambiare il testo di un elemento:

```
const titolo = document.getElementById("titolo");  
titolo.textContent = "Nuovo Titolo!";
```

- Cambiare attributi:

```
const link = document.querySelector("a");  
link.setAttribute("href", "https://www.example.com");
```

/**

- Aggiorna il contenuto HTML di un elemento specificato utilizzando la proprietà `innerHTML`.
-
- `@param {string} val` - Il testo o valore da inserire all'interno dell'elemento.
- Questo valore viene assegnato tramite la proprietà `innerHTML`, che permette di inserire sia testo semplice che markup HTML.
- Attenzione: l'utilizzo di `innerHTML` può esporre a rischi di sicurezza (XSS) se il contenuto non è opportunamente validato o sanificato.

*/

Modificare lo stile degli elementi

- Modificare lo stile:

```
const box = document.getElementById("box");  
box.style.backgroundColor = "yellow";  
box.style.fontSize = "20px";
```

Aggiungere e rimuovere elementi nel DOM

- Aggiungere un elemento figlio:

```
const lista = document.querySelector("ul");  
const nuovoElemento = document.createElement("li");  
nuovoElemento.textContent = "Nuovo elemento";  
lista.appendChild(nuovoElemento);
```

- Rimuovere un elemento:

```
const elemento = document.getElementById("daRimuovere");  
elemento.remove();
```

Gestire eventi sugli elementi

- Gestire eventi:

```
const bottone = document.getElementById("mioBottone");  
bottone.addEventListener("click", function () {  
    alert("Hai cliccato il bottone!");  
});
```

Esercitazione: Aggiunta automatica di elementi HTML

Crea una pagina HTML con una lista `<ul>` vuota e un bottone "Aggiungi elemento". Quando l'utente clicca sul bottone, aggiungi automaticamente un nuovo `<li>` alla lista con il testo "Elemento X", dove X è il numero progressivo dell'elemento.

Requisiti:

- Ogni click sul bottone aggiunge un nuovo elemento alla lista.
- Il testo di ogni elemento deve essere incrementale ("Elemento 1", "Elemento 2", ...).
- Dopo aver aggiunto un elemento, stampa in console tutti i testi degli `<li>` presenti nella lista.

Soluzione esempio

```
<ul id="lista"></ul>
<button id="aggiungi">Aggiungi elemento</button>
<script>
  const lista = document.getElementById("lista");
  const bottone = document.getElementById("aggiungi");
  let contatore = 1;

  bottone.addEventListener("click", function () {
    const nuovoLi = document.createElement("li");
    nuovoLi.textContent = "Elemento " + contatore++;
    lista.appendChild(nuovoLi);

    // Stampa tutti i testi degli <li>
    const elementi = document.querySelectorAll("#lista li");
    const testi = Array.from(elementi).map((li) => li.textContent);
    console.log(testi);
  });
</script>
```

Eventi in JavaScript

- Gli **eventi** sono azioni che accadono nella pagina web (click, pressione di tasti, caricamento, ecc.) e possono essere gestiti tramite JavaScript.
- Gli eventi più comuni sono:
 - `click` - `dblclick` - `mouseover` - `mouseout`
 - `mousedown` - `mouseup` - `mousemove` - `keydown` - `keyup` - `keypress`
 - `change` - `input` - `submit` - `focus` - `blur` - `load` - `resize`
 - `scroll` - `contextmenu`
 - `touchstart`, `touchend`, `touchmove` (mobile)

Esempio: evento click

```
const bottone = document.getElementById("mioBottone");  
bottone.addEventListener("click", function () {  
    alert("Hai cliccato il bottone!");  
});
```

Esempio: evento mouseover

```
const box = document.getElementById("box");
box.addEventListener("mouseover", function () {
    box.style.backgroundColor = "lightblue";
});
box.addEventListener("mouseout", function () {
    box.style.backgroundColor = "";
});
```

Esempio: evento keydown

```
document.addEventListener("keydown", function (event) {  
    console.log("Hai premuto il tasto:", event.key);  
});
```

Esempio: evento input

```
const input = document.getElementById("testo");  
input.addEventListener("input", function () {  
    console.log("Valore attuale:", input.value);  
});
```

Esempio: evento submit su form

```
const form = document.getElementById("mioForm");
form.addEventListener("submit", function (event) {
    event.preventDefault(); // evita il reload della pagina
    alert("Form inviato!");
});
```

Esempio: evento change su select

```
const select = document.getElementById("menu");  
select.addEventListener("change", function () {  
    console.log("Hai selezionato:", select.value);  
});
```

Esempio: evento scroll

```
window.addEventListener("scroll", function () {  
    console.log("Scroll verticale:", window.scrollY);  
});
```

Esempio: evento load

```
window.addEventListener("load", function () {  
    console.log("La pagina è stata caricata!");  
});
```


Esempio: evento focus e blur

```
const input = document.getElementById("testo");
input.addEventListener("focus", function () {
    input.style.borderColor = "green";
});
input.addEventListener("blur", function () {
    input.style.borderColor = "";
});
```

Esempio: evento contextmenu (click destro)

```
document.addEventListener("contextmenu", function (event) {  
    event.preventDefault();  
    alert("Menu contestuale disabilitato!");  
});
```

Esempio: evento touchstart (mobile)

```
const area = document.getElementById("areaTouch");  
area.addEventListener("touchstart", function () {  
    area.textContent = "Touch rilevato!";  
});
```

Nota: Puoi combinare più eventi per creare interazioni complesse e dinamiche nella tua pagina web.

Iterare e modificare più elementi HTML

- Selezionare tutti gli elementi (ad esempio tutti i `<li>`):

```
const elementi = document.querySelectorAll("li");
```

- Ciclare su tutti gli elementi e modificarli:

```
elementi.forEach(function (elemento) {  
  elemento.textContent = "Elemento modificato!";  
});
```

- Puoi anche cambiare classi, attributi o stili di ciascun elemento all'interno del ciclo.
- `forEach` funziona su `NodeList` (il risultato di `querySelectorAll`).

Esempio: aggiungere una classe a tutti i paragrafi

```
const paragrafi = document.querySelectorAll("p");  
paragrafi.forEach((p) => p.classList.add("evidenziato"));
```

Esercitazione 1: Seleziona un elemento per ID e cambia il testo

Scrivi un codice JavaScript che seleziona un elemento con `id="titolo"` e ne modifica il testo in "Titolo modificato dall'esercizio".

Soluzione Esercitazione 1

```
const titolo = document.getElementById("titolo");  
titolo.textContent = "Titolo modificato dall'esercizio";
```

Esercitazione 2: Cambia il contenuto dell'ultimo elemento di una lista

Data una lista di elementi `<li>`, scrivi un codice che seleziona l'ultimo elemento e ne cambia il testo in "Ultimo elemento aggiornato".

Soluzione Esercitazione 2

```
const elementi = document.querySelectorAll("li");  
const ultimo = elementi[elementi.length - 1];  
ultimo.textContent = "Ultimo elemento aggiornato";
```

Esercitazione 3: Aggiungi una classe a tutti i paragrafi

Scrivi un codice che seleziona tutti gli elementi `<p>` e aggiunge loro la classe CSS `"evidenziato"`.

Soluzione Esercitazione 3

```
const paragrafi = document.querySelectorAll("p");  
paragrafi.forEach((p) => p.classList.add("evidenziato"));
```

Esercitazione 4: Cambia il colore del primo elemento con una certa classe

Seleziona il primo elemento con la classe `"paragrafo"` e cambia il suo colore del testo in rosso usando JavaScript.

Soluzione Esercitazione 4

```
const primoParagrafo = document.querySelector(".paragrafo");  
primoParagrafo.style.color = "red";
```

Cambiare classi CSS con JavaScript

- Aggiungere, rimuovere o alternare classi:

```
const bottone = document.querySelector("#mioBottone");  
  
// Aggiungi una classe  
bottone.classList.add("attivo");  
  
// Rimuovi una classe  
bottone.classList.remove("attivo");  
  
// Alterna una classe (aggiunge se non c'è, rimuove se c'è)  
bottone.classList.toggle("attivo");
```

Modificare il contenuto degli elementi

- Cambiare il testo:

```
const titolo = document.getElementById("titolo");  
titolo.textContent = "Nuovo titolo!";
```

- Cambiare l'HTML interno:

```
const contenitore = document.querySelector(".contenitore");  
contenitore.innerHTML = "<p>Nuovo <strong>contenuto</strong>!</p>";
```

Focus: L'espressione regolare per la password

Nel codice di validazione password viene utilizzata questa espressione regolare:

```
const regex = /^(?=.*[A-Z])(?=.*\d){8,}$/;  
// Test della password  
console.log(regex.test("Password1")); // true  
console.log(regex.test("password")); // false
```

Vediamo cosa significa:

- `^` e `$` indicano l'inizio e la fine della stringa (la password deve rispettare tutta la regola).
- `(?=.*[A-Z])` richiede almeno una lettera maiuscola.
- `(?=.*\d)` richiede almeno una cifra numerica.

In sintesi:

La password è valida solo se contiene almeno 8 caratteri, almeno una lettera maiuscola e almeno un numero.

Esempi:

- "Password1"  valida
- "password"  non valida (manca maiuscola e numero)
- "Passw1"  non valida (meno di 8 caratteri)
- "PASSWORD1"  valida

Le espressioni regolari sono strumenti potenti per validare stringhe secondo regole precise.

Esercizio: Validazione campi input

1. Verifica corrispondenza email

Scrivi un codice JavaScript che controlla se due campi di input email (`email` e `confermaEmail`) contengono lo stesso valore. Se non corrispondono, mostra un messaggio di errore.

2. Verifica complessità password

Scrivi un codice che controlla se la password inserita rispetta questi criteri:

- Almeno 8 caratteri
- Almeno una lettera maiuscola
- Almeno un numero

3. Validazione generale dei campi

Scrivi una funzione che, al submit di un form, verifica che tutti i campi obbligatori siano compilati e che email e password siano validi secondo le regole sopra. Se ci sono errori, impedisce l'invio del form e mostra i messaggi di errore.

Soluzione: Validazione campi input

```
const form = document.getElementById("mioForm");
const email = document.getElementById("email");
const confermaEmail = document.getElementById("confermaEmail");
const password = document.getElementById("password");
const errori = document.getElementById("errori");

form.addEventListener("submit", function (event) {
  errori.innerHTML = "";
  let valido = true;

  // Verifica campi obbligatori
  if (!email.value || !confermaEmail.value || !password.value) {
    errori.innerHTML += "<p>Tutti i campi sono obbligatori.</p>";
    valido = false;
  }

  // Verifica corrispondenza email
  if (email.value !== confermaEmail.value) {
    errori.innerHTML += "<p>Le email non corrispondono.</p>";
    valido = false;
  }

  // Verifica complessità password
  const regex = /^(?=.*[A-Z])(?=.*\d){8,}$/;
  if (!regex.test(password.value)) {
    errori.innerHTML +=
      "<p>La password deve avere almeno 8 caratteri, una maiuscola e un numero.</p>";
    valido = false;
  }

  if (!valido) {
    event.preventDefault();
  }
});
```

1. Introduzione ad AJAX

- Cos'è AJAX
- Vantaggi e svantaggi

Cos'è AJAX e cosa permette di fare

- AJAX (Asynchronous JavaScript and XML) è una tecnica che consente di inviare e ricevere dati dal server in modo asincrono, senza ricaricare la pagina.
- Permette di aggiornare parti della pagina web dinamicamente, migliorando l'esperienza utente.
- Non è un linguaggio, ma una combinazione di tecnologie: JavaScript, oggetti XMLHttpRequest o Fetch API, HTML/CSS e spesso dati in formato JSON.

Approfondimento: Il formato JSON

- **JSON** (JavaScript Object Notation) è un formato leggero per lo scambio di dati, molto usato nelle comunicazioni AJAX.
- È facile da leggere e scrivere sia per le persone che per le macchine.
- In JavaScript, puoi convertire un oggetto in JSON con `JSON.stringify()` e viceversa con `JSON.parse()`.

Esempio di oggetto JavaScript:

```
const persona = {  
  nome: "Mario",  
  eta: 30,  
};
```

Conversione in JSON:

```
const json = JSON.stringify(persona);  
// Risultato: '{"nome":"Mario","eta":30}'
```

Parsing da JSON a oggetto:

```
const oggetto = JSON.parse(json);  
// Risultato: { nome: "Mario", eta: 30 }
```

- JSON supporta tipi semplici: stringhe, numeri, booleani, array, oggetti e `null`.
- È il formato più usato per scambiare dati tra client e server nelle applicazioni web moderne.
- Attenzione: le chiavi e le stringhe devono essere racchiuse tra doppi apici

Esempio pratico: file `.json` e endpoint di test

Esempio di file `dati.json` :

```
{  
  "nome": "Anna",  
  "cognome": "Rossi",  
  "eta": 28,  
  "hobby": ["lettura", "sport", "viaggi"]  
}
```

Come caricare un file JSON da JavaScript (con Fetch API):

```
fetch("dati.json")  
  .then((response) => response.json())  
  .then((dati) => {  
    console.log("Nome:", dati.nome);  
    console.log("Hobby:", dati.hobby.join(", "));  
  })  
  .catch((error) => console.error(error));
```

CORS e Live Server in VS Code

Cos'è CORS?

- **CORS** (Cross-Origin Resource Sharing) è una politica di sicurezza dei browser che limita le richieste HTTP tra siti diversi (origini diverse).
- Se una pagina web prova a richiedere dati da un dominio diverso, il browser blocca la richiesta a meno che il server non consenta esplicitamente l'accesso tramite intestazioni CORS.

Esempio di errore CORS:

Access to fetch at 'https://api.esterno.com/dati' from origin 'http://localhost:5500' has been blocked by CORS policy

Live Server in VS Code

- **Live Server** è un'estensione di VS Code che avvia un server locale per testare le pagine web.
- Serve i file tramite `http://localhost:5500` (o altra porta), simulando un ambiente reale.
- Utile per evitare problemi di CORS quando si lavora in locale: molte API pubbliche consentono richieste da `localhost`.

Come usare Live Server:

1. Installa l'estensione "Live Server" da VS Code.
2. Clicca con il tasto destro su un file HTML e scegli "Open with Live Server".
3. La pagina si aprirà nel browser su `localhost`.

Endpoint pubblici di test per provare JSON nel browser

Puoi provare a fare richieste AJAX a questi endpoint gratuiti:

- <https://jsonplaceholder.typicode.com/users/1>
- <https://jsonplaceholder.typicode.com/posts>

Esempio:

```
fetch("https://jsonplaceholder.typicode.com/users/1")
  .then((res) => res.json())
  .then((user) => {
    console.log("Nome utente:", user.name);
    console.log("Email:", user.email);
  });
```

Puoi incollare questo codice direttamente nella console del browser per vedere il risultato.

Cosa si può fare con AJAX

- Caricare dati da un server e visualizzarli senza ricaricare la pagina.
- Inviare dati di un form in background.
- Aggiornare solo una sezione della pagina (es. risultati di ricerca, notifiche).
- Costruire applicazioni web interattive come chat, dashboard, mappe dinamiche.

Esempio base: richiesta GET con Fetch API

```
fetch("https://jsonplaceholder.typicode.com/posts/1")  
  .then((response) => response.json())  
  .then((dati) => {  
    console.log("Titolo:", dati.title);  
  });
```

- In questo esempio, viene richiesta una risorsa da un server e il titolo viene stampato in console.

Esempio base: invio dati con POST

```
fetch("https://jsonplaceholder.typicode.com/posts", {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({ title: "Nuovo post", body: "Testo", userId: 1 }),  
})  
  .then((response) => response.json())  
  .then((dati) => {  
    console.log("Risposta dal server:", dati);  
  });
```

- Questo esempio mostra come inviare dati a un server e gestire la risposta.

Struttura di una richiesta AJAX

- XMLHttpRequest vs Fetch API
- Sintassi base

Struttura di una richiesta AJAX

XMLHttpRequest

- È il metodo "storico" per effettuare richieste AJAX.
- Richiede più codice e gestione manuale degli stati della richiesta.

Esempio base:

```
const xhr = new XMLHttpRequest();
xhr.open("GET", "https://jsonplaceholder.typicode.com/posts/1");
xhr.onload = function () {
  if (xhr.status === 200) {
    const dati = JSON.parse(xhr.responseText);
    console.log("Titolo:", dati.title);
  }
};
xhr.send();
```

Fetch API

- Introdotta nei browser moderni, è più semplice e basata sulle Promise.
- Sintassi più pulita e leggibile.

Esempio base:

```
fetch("https://jsonplaceholder.typicode.com/posts/1")  
  .then((response) => response.json())  
  .then((dati) => {  
    console.log("Titolo:", dati.title);  
  });
```

- Gestione più semplice degli errori e delle risposte asincrone.
- Supporta anche le richieste POST, PUT, DELETE, ecc.

Introduzione alle Promise in JavaScript

- Una **Promise** è un oggetto che rappresenta il risultato futuro di un'operazione asincrona.
- Permette di gestire codice asincrono in modo più leggibile rispetto alle callback tradizionali.
- Una Promise può essere in tre stati:
 - **pending** (in attesa)
 - **fulfilled** (risolta)
 - **rejected** (rifiutata)

Sintassi base delle Promise

Creazione di una Promise:

```
const promessa = new Promise((resolve, reject) => {  
  // Operazione asincrona  
  if (/* tutto ok */) {  
    resolve('Risultato');  
  } else {  
    reject('Errore');  
  }  
});
```

Utilizzo di then/catch:

```
promessa
  .then((risultato) => {
    console.log("Successo:", risultato);
  })
  .catch((errore) => {
    console.error("Errore:", errore);
  });
```

- Le Promise sono usate da molte API moderne, come la Fetch API.
- Permettono di concatenare operazioni asincrone in modo chiaro e gestire facilmente gli errori.

Esercizio riepilogativo: Mini-app di gestione lista

Crea una semplice pagina HTML con una lista di elementi `<ul>` e un campo di input con un bottone "Aggiungi". Scrivi il codice JavaScript che:

1. Permette all'utente di inserire un nuovo elemento nella lista digitando il testo nell'input e cliccando il bottone.
 2. Evidenzia tutti gli elementi della lista con uno sfondo giallo quando vengono cliccati.
 3. Recupera tutti gli elementi della lista e li stampa in console come array di stringhe.
- Usa `addEventListener` per gestire il click sul bottone e sugli elementi della lista.